

Práctica 8: Módulos secuenciales - Registros y contadores

Objetivo

Aplicar el uso de registros y contadores en el diseño de sistemas secuenciales síncronos.

Se trabajará en los circuitos:

- **Ej 8.1: Register**
- **Ej 8.2: Ring cnt**
- **Ej 8.3: CntMOD 25**
- **Ej 8.4: Lavadora**

Módulos secuenciales

En la librería de elementos de memoria (*Memory*) no solo se proporcionan los diferentes tipos de biestables. Además, dicha librería proporciona módulos más complejos que implementan las funcionalidades más frecuentes de los sistemas secuenciales.

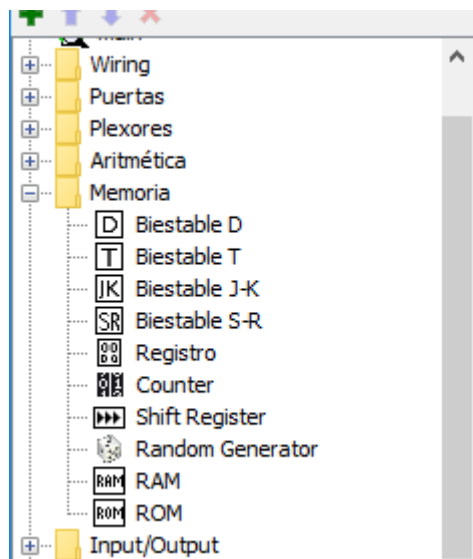


Figura 8.1: Componentes de la librería Memoria.

A continuación se describe el funcionamiento de los módulos proporcionados por la librería y su relación con los tipos de biestables existentes.

Registros

Los registros se implementan con biestables tipo D. Logisim proporciona dos tipos de registros:

- **Registros de almacenamiento** (*Register*), y
- **Registros de desplazamiento** (*Shift Register*).

Registro de almacenamiento

Es un elemento de memoria multibit que almacena el valor de su entrada de datos (D) cuando se activa la señal de disparo. A su vez consta de entrada de habilitación (en) y de borrado asíncrono (puesta a 0).

Su funcionamiento es equivalente al de una batería de biestables D conectados en paralelo, de modo que cada biestable almacena un bit de datos. Si el registro está inhabilitado con señal en=0, el registro no modifica su estado.

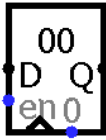


Figura 8.2: Símbolo Logisim para un registro.

Los registros de almacenamiento se emplean para guardar información relacionada con datos temporales usados en los sistemas digitales. Por ejemplo, resultados intermedios obtenidos en operaciones aritméticas. También se emplean para sincronizar un flujo de datos obtenidos de forma asíncrona.

Ejercicio 8.1: Registro de almacenamiento

- Completa el circuito "Ej 8.1: Registro" (ver Fig. 8.3).
- Comprueba el comportamiento de un registro de almacenamiento de 4 bits empleando con el circuito completo. Prueba la modificación del dato almacenado en el registro.
- Emplea un tick de simulación de 2 Hz y un pulsador para el reinicio asíncrono del registro.

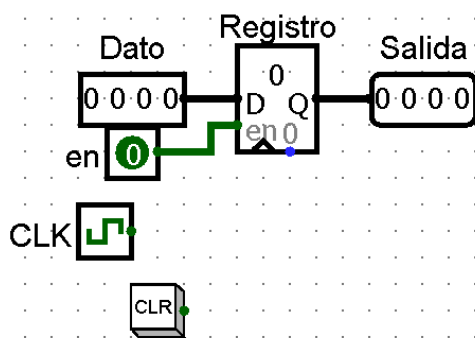


Figura 8.3: Esquema de circuito empleado como pista del ejercicio 8.1.

Registro de desplazamiento (Shift Register)

Es un registro multibit (multietapa) similar al anterior, pero con cada flanco de sincronismo se desplaza el contenido del registro un bit hacia la derecha. Este funcionamiento con biestables D se obtendría conectándolos en cascada de modo que la salida de una etapa fuese la entrada de la siguiente. Al igual que el registro anterior cuenta con entradas para sincronismo, carga de la primera etapa, inhibición, borrado asíncrono y carga en paralelo simultánea de bits en cada etapa. La salida del registro (acceso a sus bits) puede ser tanto en serie (bit en la etapa final) como en paralelo.

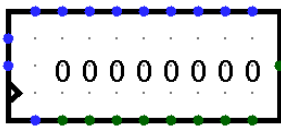


Figura 8.4: Símbolo Logisim para un registro de desplazamiento.

Los registros de desplazamiento se emplean para hacer la conversión de flujos de datos entre paralelo <-> serie (en ambos sentidos).

Inicializados del modo oportuno y con realimentación desde su etapa final a la inicial, permiten obtener secuencias equivalentes a las producidas mediante decodificación de un contador. Dichos circuitos se denominan contadores basados en registros de desplazamiento, que pueden ser de dos tipos:

- **Contador en anillo.** Si se inicializa una de las etapas con un valor diferente al resto. Por ejemplo, un contador en anillo de 4 bits genera la secuencia: 1000, 0100, 0010, 0001, 1000, ... Dependiendo de que el bit diferente al resto sea el 1 o el 0, la codificación obtenida se denomina *one-hot* y *one-cold bit encoding*, respectivamente.

- **Contador Johnson.** Si todas las etapas se inicializan a 0 y el valor negado de la última etapa realimenta a la primera. La secuencia generada para 4 bits sería: 0000, 1000, 1100, 1110, 1111, 0111, 0011, 0001, 0000, ...

Ejercicio 8.2: Contador en anillo

- Completa el circuito "Ej 8.2: Ring cnt" para reproducir el comportamiento de un contador en anillo de 4 bits con *one-bit encoding* empleando del módulo *Shift Register* de Logism.
- Emplea un separador de bits y una constante multibit para inicializar el registro.
- Realimenta e inicializa el circuito del modo apropiado.
- Simula el comportamiento del circuito resultante y comprueba su correcto funcionamiento.
- Haz una copia del circuito y modifícala para que implemente un contador Johnson. Ten en cuenta que quizás debes eliminar algunos elementos.

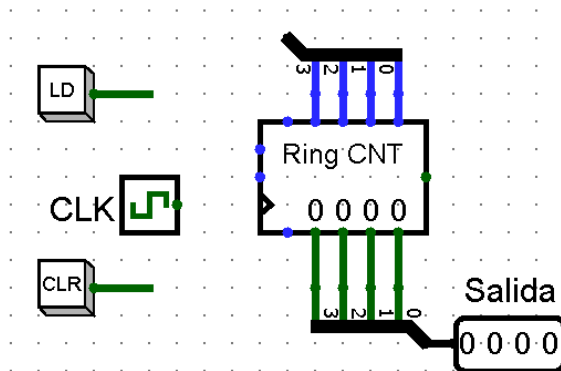


Figura 8.5: Esquema de circuito empleado como pista del ejercicio 8.2.

Contador

Un contador es un elemento de memoria cuyo propósito es contar. Se pueden contar multitud de eventos, e incluso cuando se cuentan los flancos de una señal de reloj estamos ante un elemento de cuenta de tiempo. Por tanto, los contadores permiten la implementación de acciones temporizadas.

Los contadores se caracterizan por generar una secuencia única de salida sin valores repetidos, que puede ser progresiva (ascendente) o regresiva (descendente). Son módulos secuenciales que emplean biestables para memorizar los valores de cuenta (estados). Los biestables empleados suelen ser los de tipo T, pero es posible emplear cualquier otro tipo de biestable modificando apropiadamente el diseño.

Puesto que un biestable permite 2 estados posibles, n biestables permiten un máximo de 2^n estados. Por ejemplo, con un contador ordenado ascendente de 4 bits, el valor máximo de cuenta es 15 (se comienza en 0). Si el valor máximo necesario es inferior a 15, pero superior a 7. Se dice que el contador es incompleto o truncado.

El número de estados de un contador se indica señalando que el contador es módulo (MOD) x , siendo x el número de estados. Así, el contador truncado más popular de 4 bits es el MOD 10 o *contador de décadas o BCD*, que posee un valor máximo de cuenta de 9.

La biblioteca de memoria (*Memory*) de Logism proporciona un elemento contador (*Counter*) con una funcionalidad muy flexible.

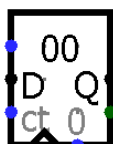


Figura 8.6: Símbolo Logism para un contador.

Este elemento incrementa o decrementa su estado con cada flanco de sincronismo. Sus atributos son:

- Número de bits (*data bits*). Equivale al número interno de biestables empleados y condiciona el valor máximo de cuenta.
- Valor máximo (*maximum value*). Valor máximo de cuenta expresado en hexadecimal, como máximo será $2^n - 1$, con n el número de bits.
- Acción al pasar a desbordamiento (*action on overflow*). Comportamiento del contador cuando se llega al fin de cuenta.
 - *Wrap around*. El siguiente valor es 0 cuando el contador es ascendente y el valor máximo cuando es descendente.
 - *Stay at value*. El contador se mantiene en el valor máximo o en 0 si es descendente.
 - *Continue counting*. Sigue contando hasta llegar al valor máximo determinado por el número de bits.
 - *Load next value*. Carga el siguiente valor que exista en la entrada D.
- Modalidad de disparo (*trigger*). Flanco de subida o de bajada.

Además de la entrada de sincronismo, tiene entradas para:

- Control del sentido de la cuenta,
- Carga de un dato (D), y
- Reinicio asíncrono.

Sus salidas son el valor de la cuenta en binario y un bit (*terminal count*) que indica si se alcanzó el fin de cuenta.

Ejercicio 8.3: Contadores concatenados y temporización de acciones

- Completa el circuito "Ej 8.3: CntMOD 25".
- Observa como la conexión del biestable D memoriza la pulsación del inicio de cuenta.
- Configura el biestable D y el contador con disparo por nivel alto.
- Configura el contador y la frecuencia de tick de simulación para que el conjunto realice una cuenta de 25 segundos y comprueba que el funcionamiento es el deseado.
- Configura la sonda de prueba para mostrar el valor decimal de la cuenta en segundos.
- Conecta los elementos necesarios para que al llegar a los 25 s, el sistema detenga la cuenta y quede detenido en este valor.
- Conecta un LED que se mantenga activado durante los 25 s de duración de la cuenta y se desactive al finalizar la cuenta. Esto es, una *acción temporizada con retardo a la desactivación*.

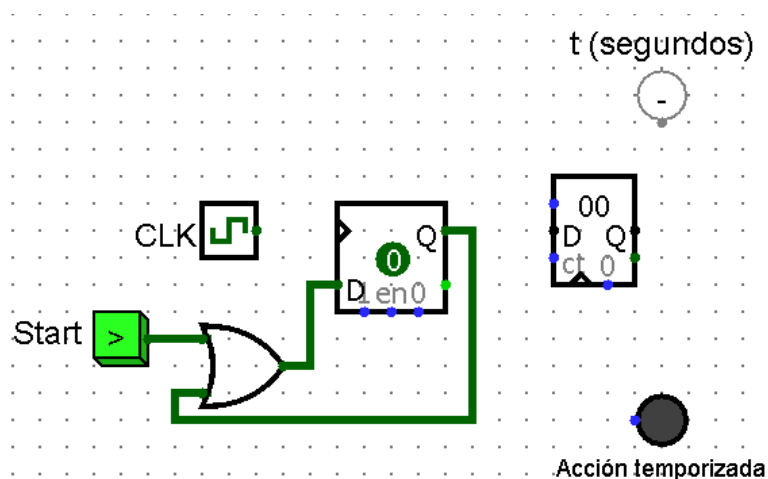


Figura 8.7: Esquema de circuito empleado como pista del ejercicio 8.3.

La salida de los contadores se pueden emplear como entradas de otros módulos combinacionales o secuenciales para modificar el código binario final generado. Algunos ejemplos son:

- cuando la secuencia requerida precisa un número de bits superior a los necesarios para codificar los estados,
- cuando la secuencia deseada tiene valores repetidos o desordenados, etcétera.

En los denominados secuenciadores se precisa activar y desactivar un número determinado de subsistemas en una determinada secuencia de pasos. Por ejemplo, en electrodomésticos que siguen un determinado "programa". En estos casos un contador permite temporizar las fases del programa, obteniendo en cada fase el valor de una secuencia con los dispositivos que deben estar activados (1) y desactivados (0).

Memoria ROM

Permite obtener los valores almacenados en una memoria direccionada. Sus atributos son:

- Ancho de bus de direcciones (*Address Bit Width*). Determina el número de posiciones de memoria disponibles.
- Tamaño de los datos (*Data Bit Width*). Tamaño de los datos almacenados en cada posición de memoria.
- Contenido de la memoria. Despliega una nueva ventana en la que se editan los valores de los datos en memoria, expresados en hexadecimal.

Este elemento combinado con los contadores permite obtener secuencialmente los valores almacenados en la memoria. Este método es muy flexible, ya que permite emplear el mismo circuito para generar distintas secuencias cambiando únicamente el contenido de la memoria.

Ejercicio 8.4: Secuenciador con memoria ROM

- Añade el circuito "**Ej 8.4: Lavadora**" en el que se simulará la lógica de activación simplificada de las fases de una lavadora sencilla.



La lavadora cuenta con 13 fases o estados en los que se accionarán 4 subsistemas (salidas):

- carga de agua (0001) -> 1,
 - giro de tambor (0010) ->2,
 - desagüe (0100)->4, y
 - toma de detergente (1000)->8.
- Si tenemos en cuenta la codificación de salidas mencionada anteriormente, la secuencia de fases de lavado que se debe seguir es: 1,2,4,9,2,4,1,2,4,1,2,4,2. Observa que el código 9 corresponde a la toma de detergente simultánea a la carga de agua.
 - Ten en cuenta que en el inicio y final de un programa todas las salidas deben estar desactivadas (0000) -> 0.
 - Configura la memoria ROM para obtener el resultado deseado. Intenta el reinicio síncrono del contador para que la última fase tenga la duración deseada y añade un pulsador de parada asíncrona.
 - Ajusta la frecuencia del reloj para que cada fase dure 2 s y considera todas las fases de idéntica duración. La implementación de fases de distinta duración se conseguiría repitiendo los valores de secuencia en fases sucesivas generadas por el contador.

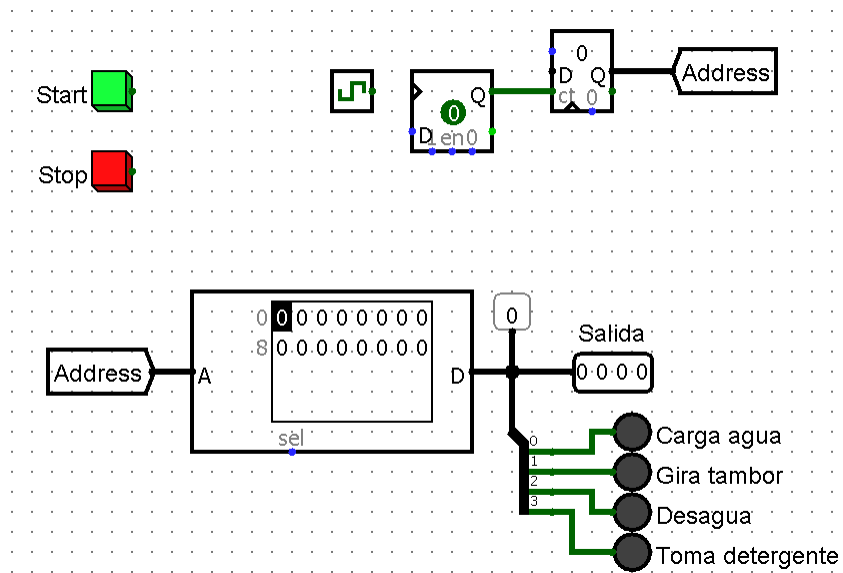


Figura 8.11: Esquema de circuito empleado como pista del ejercicio 8.4.

NOTA: En una lavadora real la parada debe estar más controlada pues deberíamos poder pausar el programa de lavado y reanudarlo en la fase deseada. Puedes reflexionar en cómo conseguir esta funcionalidad mediante ajuste del valor de carga del contador (D).

Entregable

- Salva el proyecto donde has ido realizando los ejercicios propuestos y súbelos a la tarea asociada a esta práctica. Nombra al fichero "<GrupoPr_Apellido_InicialNombre>_Pr8.circ".